

Pipeline ETL — Clima (INMET) e Municípios (IBGE)

Sprint 1 do TCC: identificação e predição de superlotação de hospitais públicos causada por surtos de dengue

Parte 1 — Como explicar o projeto

O que é, em uma frase

“Um pipeline automatizado que busca dados de clima (INMET) e de município/população (IBGE), limpa e organiza esses dados, e guarda tudo num banco PostgreSQL — para depois cruzar com casos de dengue e prever superlotação hospitalar.”

O que o projeto entrega (Sprint 1)

Item	O que é
Pipeline ETL	Um programa Python que faz 3 etapas: Extract (busca dado bruto na fonte), Transform (limpa/valida), Load (grava no banco)
Banco PostgreSQL estruturado	5 tabelas prontas, com relações entre si (chaves estrangeiras)
Rotina automática diária	Um agendador que roda o pipeline sozinho, todo dia, sem precisar de ninguém apertar botão
Prevenção de duplicidade	Rodar o pipeline várias vezes não duplica dado nenhum no banco
Preparação para o futuro	Já existe uma “gaveta vazia” pronta para quando a API da Prefeitura do Recife (casos de dengue) for integrada

De onde vêm os dados

1. **IBGE** — nome do município, UF, região, população estimada, área territorial, e a densidade demográfica (calculada por nós: população ÷ área). Hoje focado em Pernambuco (185 municípios).
2. **INMET** — clima diário (temperatura máxima/mínima/média, chuva total do dia, umidade, vento, pressão), desde 2021, das estações meteorológicas de PE. Vem de um arquivo oficial que o próprio INMET disponibiliza (pacotes anuais), depois que descobrimos que a API “em tempo real” que tentamos primeiro tinha sido descontinuada.

As tabelas que o banco gera

Tabela	O que guarda
municipios	Um município por linha: nome, UF, população, área, densidade
estacoes_inmet	Uma estação de clima por linha: nome, localização, situação
clima_diario	Um dia + uma estação por linha: temperatura, chuva, umidade etc
casos_dengue	Já criada, mas ainda vazia — vai receber os casos de dengue na próxima etapa
etl_execution_log	Um registro por execução do pipeline: quando rodou, quantos dados trouxe, se deu certo

Por que não duplica dado

Cada tabela tem uma regra no banco (chamada UNIQUE CONSTRAINT) dizendo “não pode ter duas linhas com a mesma estação + mesmo dia”, por exemplo. Quando o pipeline roda de novo, ele tenta inserir tudo, mas o banco automaticamente ignora o que já existe — só entra o que é novo.

Dificuldade encontrada e corrigida no caminho A Sprint 1 entrega a arquitetura completa, o banco estruturado, e o pipeline do IBGE já validado com dados reais. O pipeline do INMET teve um imprevisto — a API que a documentação indicava tinha sido descontinuada pelo próprio instituto — e tivemos que trocar para a fonte de dados oficial de arquivos históricos. Essa correção já está feita, e funcionando.

Parte 2 — Passo a passo para rodar ao vivo

Isso é o roteiro exato para seguir na apresentação, comando por comando, mostrando funcionando.

Preparação

1. Extraia o zip `etl-dengue-clima-ibge-limpo.zip` em um lugar fácil, por exemplo:

```
C:\Users\contriadd\Desktop\apresentacao-tcc
```

2. Abra o PowerShell dentro dessa pasta (Explorador de Arquivos → dentro da pasta → digite "powershell" na barra de endereço → Enter)

3. Rode, um de cada vez:

```
python -m venv venv
```

```
venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

4. Confirme que o `.env` já está com sua senha real do PostgreSQL:

```
notepad .env
```

(confira a linha `DB_PASSWORD=` — se já estiver certa, só feche) (a senha é **261201**)

5. Teste a conexão com o banco:

```
python -m src.database.connection
```

Deve aparecer: “Conexão com o PostgreSQL estabelecida com sucesso.”

6. Crie as tabelas (só precisa fazer uma vez):

```
python main.py --setup-db --pipeline ibge
```

Reprodução

Mostre o comando principal rodando:

```
python main.py --pipeline all
```

Enquanto roda, explique o que está aparecendo na tela:

- As linhas INFO mostram cada etapa (buscando municípios, buscando estações, etc.)
- Você pode dizer: “aqui ele está buscando os 185 municípios de Pernambuco no IBGE”, “aqui está calculando a densidade demográfica”, “aqui está indo no banco inserir, evitando duplicar”

Depois que terminar, mostre os dados dentro do banco (abra outra aba/janela do PowerShell, sem fechar a que rodou o pipeline):

```
psql -U postgres -d dengue_db -c "SELECT nome, uf, populacao_estimada, densidade_demografica FROM dengue.municipios LIMIT 10;"
```

```
psql -U postgres -d dengue_db -c "SELECT * FROM dengue.clima_diario ORDER BY data_medicao DESC LIMIT 10;"
```

```
psql -U postgres -d dengue_db -c "SELECT pipeline_name, status, registros_inseridos, iniciado_em FROM dengue.etl_execution_log ORDER BY iniciado_em DESC LIMIT 5;"
```